

Средства симуляции ЦП и ОС

Лекция №14

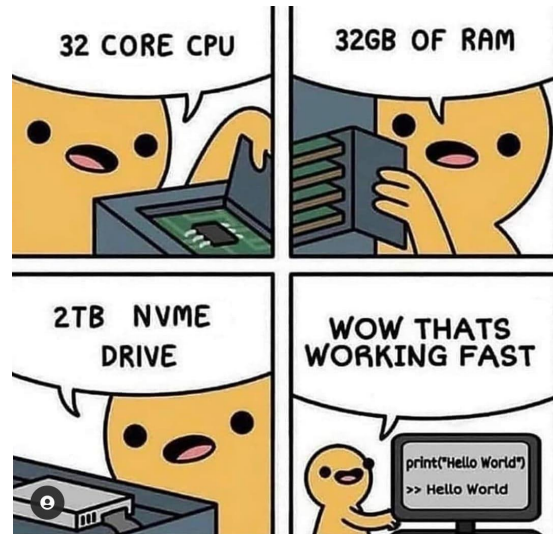
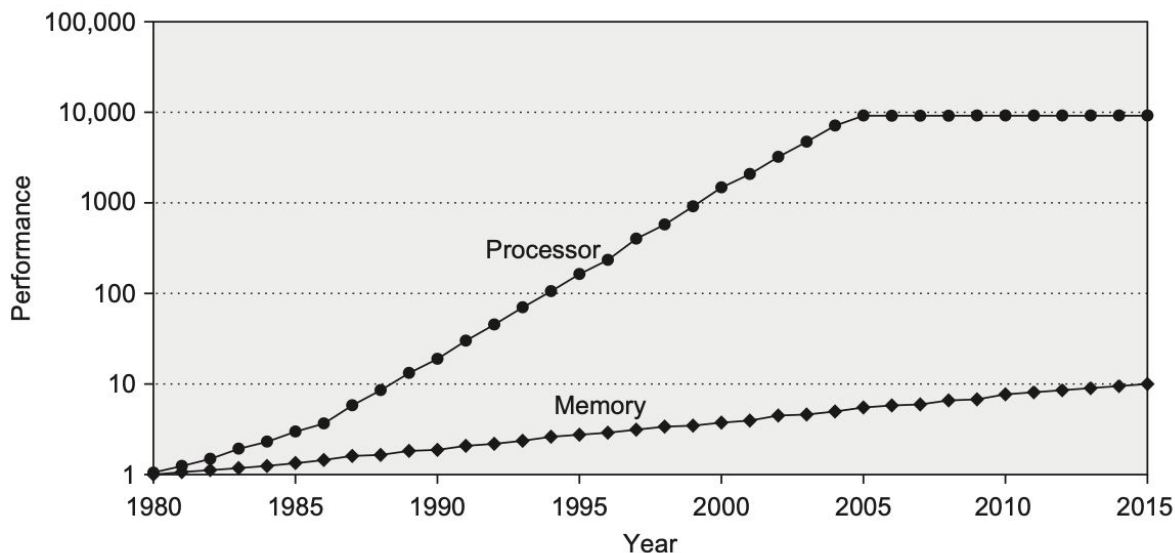


Державин Андрей
Шурыгин Антон

Стена памяти

Стена памяти

- Долгое время микропроцессоры стремительно ускорялись
- Чего не скажешь про память...

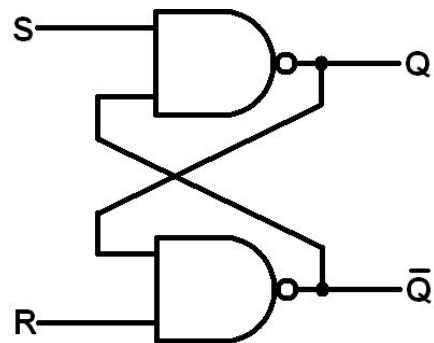


Медленная память

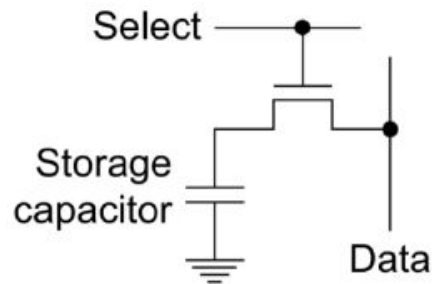
- SRAM быстрее, но дороже
- Давайте использовать SRAM везде?
- Проблема не в скорости ячейки

latency ~ size

SRAM

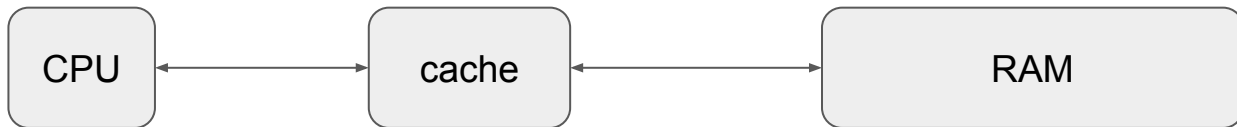


DRAM



Ускоримся?

- Добавим промежуточную быструю память
- Две области применения
 - **Частичное** решение проблемы “стены памяти”
 - Быстрая синхронизация многопроцессорных систем



Сверхоперативная память

- Локальности:
 - Временная
 - Пространственная
- Идея: храним наиболее “нужные” данные
 - Как определить “нужность”?
- Два способа попаданий данных в кэш
 - первое обращение
 - prefetching

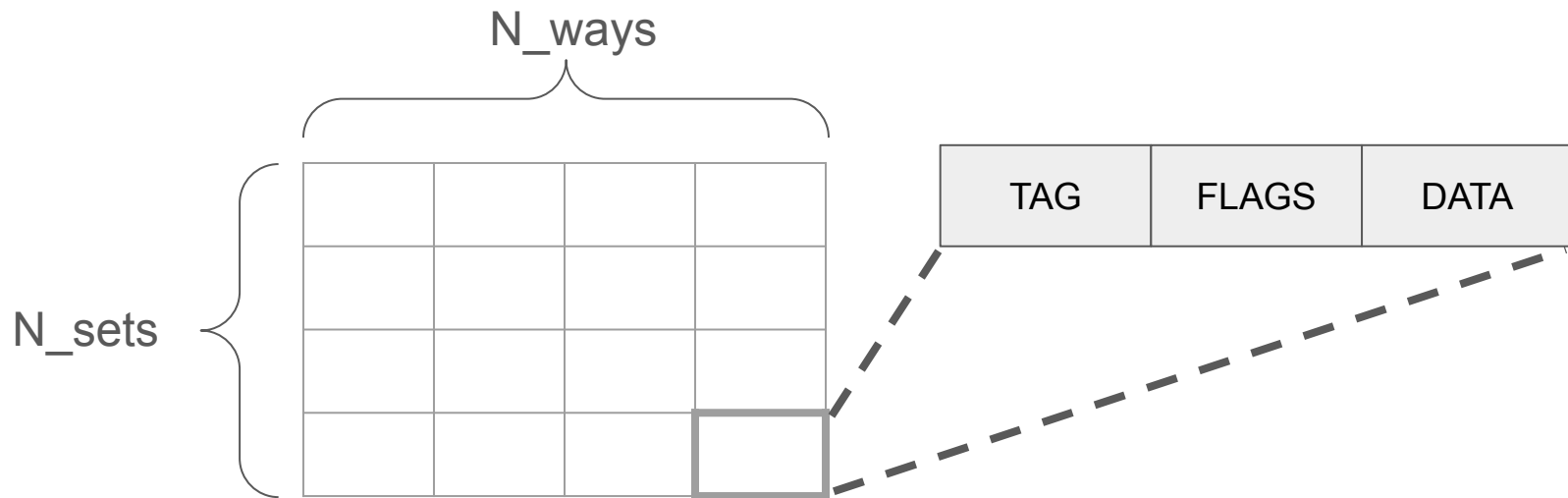
```
for (int i = 0; i < size; ++i) {  
    foo(data[i]);  
}
```

```
for (int i = 0; i < size; ++i) {  
    foo(*ptr + i);  
}
```

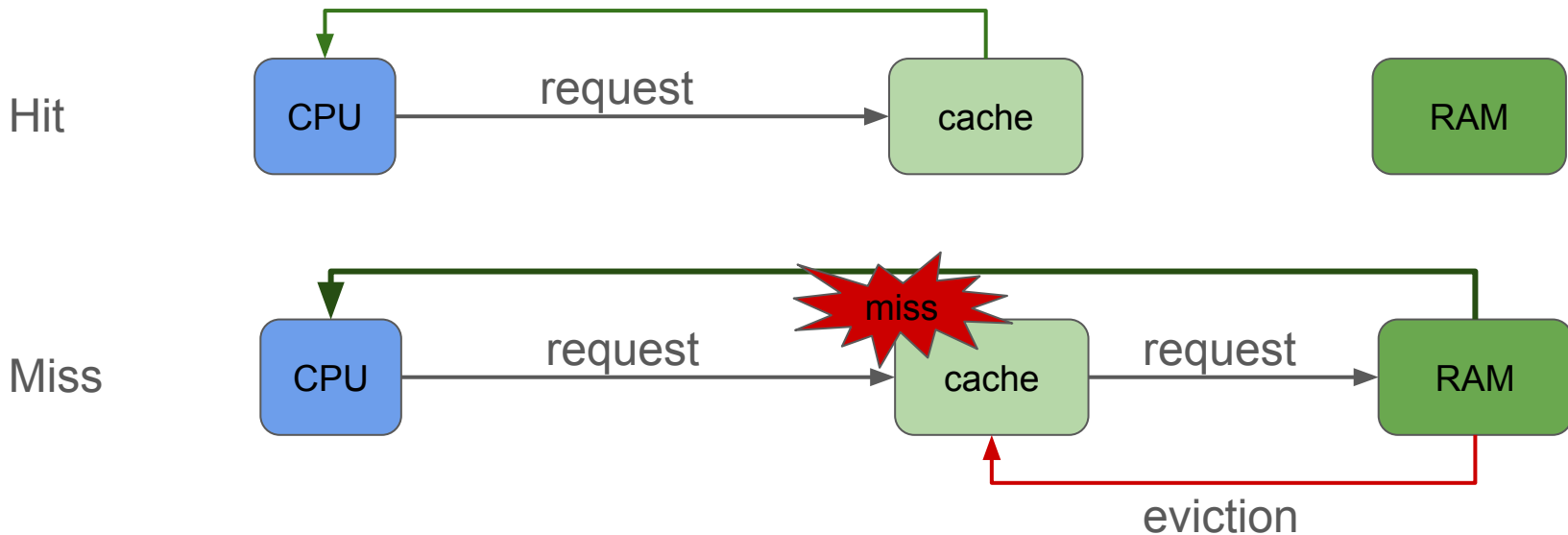
Организация кэша

Устройство кэша

- Рассмотрим общий принцип организации кэш-памяти
- Единица хранимых данных - *кэш-линия* (обычно 32/64 байта)



Устройство кэша



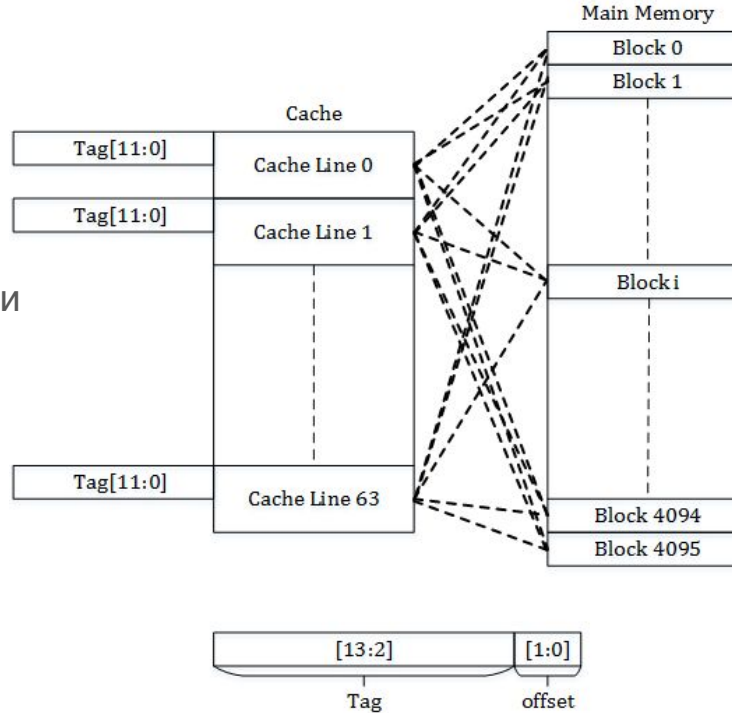
- eviction определяется **политикой замещения**
- Иногда требуется сделать сброс кэша - **flush**

Устройство кэша. Параметры

- Рассмотрим параметры обобщённого кэша.
- Размер одной линии данных - S_{line}
- Ассоциативность - N_{ways} . Определяет число способов разместить одну и ту же линию в кэше
 - $N_{ways} == 1$ - Кэш прямого отображения (*direct mapped cache*)
- N_{sets} - число сетов
 - $N_{sets} == 1$ - Полностью ассоциативный кэш (*fully associative cache*)
- Ёмкость кэша. Выражается из вышеуказанных величин
 - $C = N_{ways} * N_{sets} * S_{line}$

Устройство кэша. Fully-associative cache

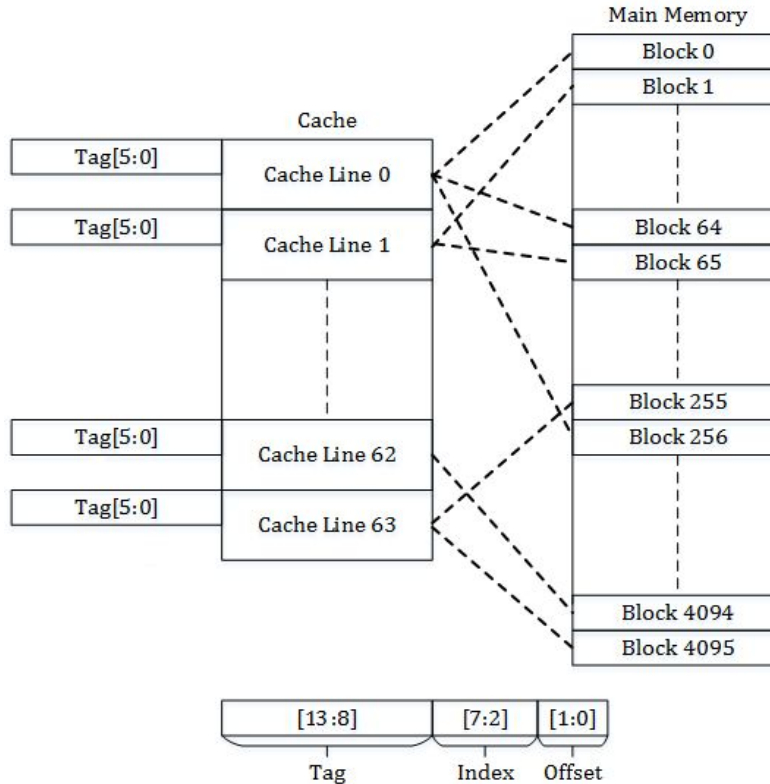
- Линия может быть в любом месте кэша
 - Поиск по **tag**
- Адрес делится:
 - **tag**
 - **offset** - смещение внутри линии
- Плюсы:
 - Высокая утилизация
 - Высокий hit rate
 - Гибкость в выборе политики
- Минусы:
 - Энергопотребление
 - Дорого



Memory Size = 16Kbytes
Memory Block Size = 4 bytes
Cache Size = 256 bytes
Block Size = 4 bytes
Number of Cache Lines = 64

Устройство кэша. Direct-mapped cache

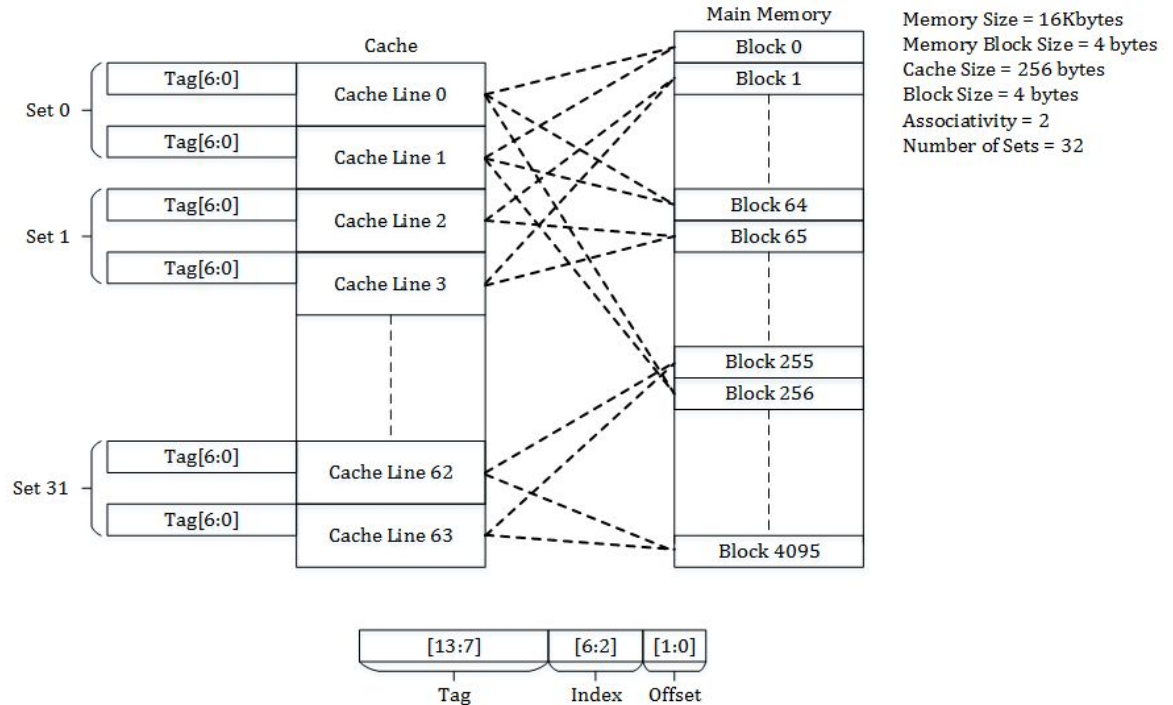
- Линия размещается внутри set
 - Поиск по **tag** внутри set
- Адрес делится:
 - **tag** - ключ
 - **set** - индекс entry
 - **offset** - смещение внутри линии
- Плюсы:
 - Энергоэффективность
 - Простота
 - Почти бесплатно
- Минусы:
 - Частые конфликты
 - Thrashing
 - Низкая утилизация



Memory Size = 16Kbytes
Memory Block Size = 4 bytes
Cache Size = 256 bytes
Block Size = 4 bytes
Associativity = 1
Number of Sets = 64

Устройство кэша. Set-associative cache

- Адрес делится:
 - **tag** - ключ
 - **set** - индекс entry
 - **offset** - смещение внутри линии
- Каждый **set** делится на **ways**
- Плюсы:
 - Компромисс
 - Hit Rate +
 - Conflicts -
 - Политики замещения
- Минусы:
 - Возможна низкая утилизация
 - Сложность
 - Скорость
 - Энергопотребление



Устройство кэша. Политики вытеснения

- Что делать, если произошел промах?
 - Выбрать свободную/невалидную линию и поместить новые данные
- Что делать, если нет свободных мест?
 - Нужно выбрать линию для вытеснения
- Как выбрать ячейку для вытеснения?
 - Существуют разные подходы
- Какая политика вытеснения считается “Идеальной”?
 - Кэш Белادي - вытесняем линию, к которой дольше всего не будет обращений
- К сожалению, мы не можем заглянуть в будущее
- Опираемся на историю обращений для принятия выбора линии для вытеснения

Устройство кэша. Политики вытеснения

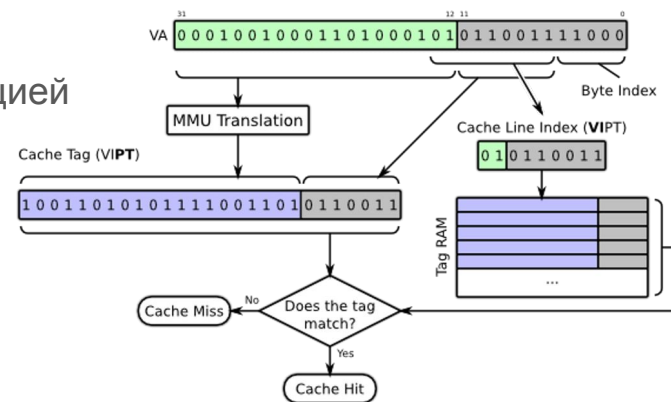
- Вытеснять всегда первую ячейку
 - Подходит только для direct mapped cache
- Вытеснять случайную ячейку
 - Может давать не оптимальные результаты
- FIFO - First In, First Out
 - Очередь внутри сэта, вытесняется самая старая ячейка внутри сэта
 - Требуется потоковое чтение памяти
- LRU - least recently used
- LFU - least frequently used
- MRU - most recently used
 - Когда бывает полезна?
- Много других политик...

Устройство кэша. Трансляция адресов

- В современных процессорах используется абстракция виртуальной памяти
- Поиск в кэше может осуществляться по физическому, виртуальному или комбинации адресов
- При использовании виртуальных адресов следует учитывать следующие факторы:
 - Задержки - трансляция адреса требует некоторое время. Для ускорения процесса могут использоваться TLB, хранящие результаты трансляций
 - Наложение - несколько виртуальных адресов могут попадать в один физический
 - Проверять, что только одна линия с данным физ. адресом лежит в кэше

Устройство кэша. Адресация

- 4 вида адресации в кэшах:
- PIPT (Physically indexed, physically tagged)
 - простые, но медленные, нет aliasing
- VIVT (Virtually indexed, virtually tagged)
 - Быстрые, но есть aliasing и гомонимы (один виртуальный адрес на много физ.)
- VIPT (Virtually indexed, physically tagged)
 - Можно искать кэш линию одновременно с трансляцией
 - Обнаружение гомонимов
- PIVT (Physically indexed, virtually tagged)
 - не дают существенных преимуществ
 - чисто академический интерес



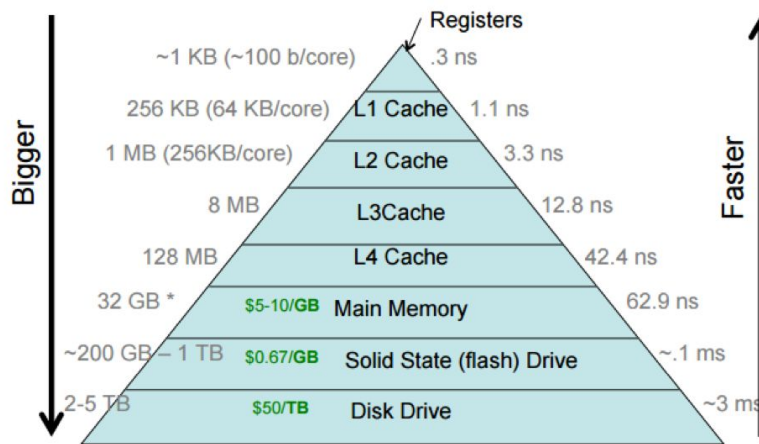
Устройство кэша. Иерархия

- Давайте увеличим N_ways и N_sets чтобы вместить как можно больше линий
- Все ли хорошо?
 - К сожалению, площадь на кристалле и допустимое энергопотребление ограничены



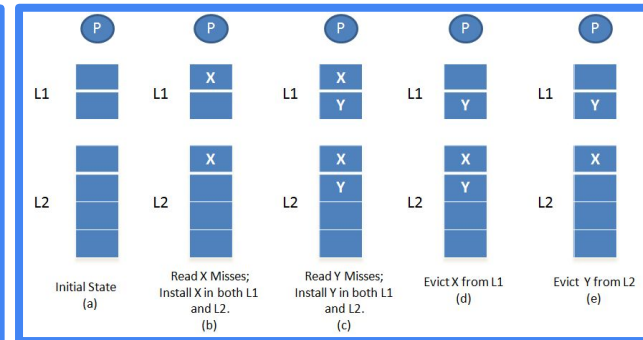
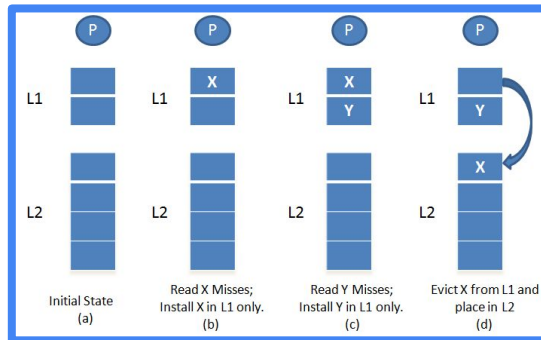
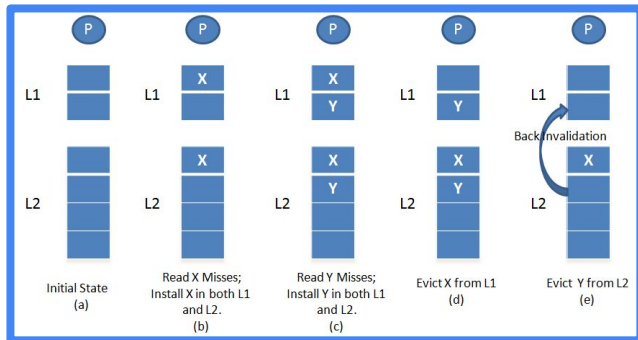
Устройство кэша. Иерархия

- Нельзя разместить большой кэш на кристалле
- Организуем кэш второго уровня L2
 - Располагается вне CPU => больше задержка
 - Имеет больший размер
- Данный подход можно масштабировать на следующие уровни



Устройство кэша. Иерархия

- Может ли одна линия находиться в нескольких кэшах?
- Есть 3 варианта
 - $L1 \subset L2$ - *инклюзивные кэши*
 - $L1 \cap L2 = \emptyset$ - *эксклюзивные кэши*
 - Ни то, ни другое - *non-inclusive non-exclusive (NINE)*



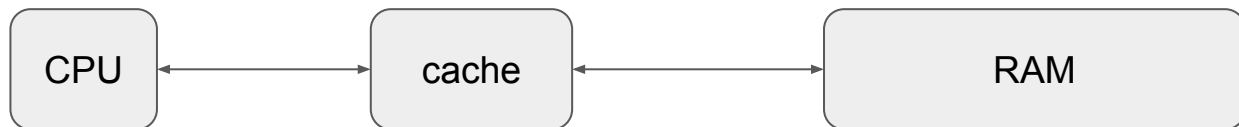
Устройство кэша. Кэш кода и данных

- В архитектуре фон-Неймана код и данные хранятся в одном пространстве памяти
- Множество используемых адресов, а также паттерны доступа к ним различаются у данных и кода
 - Может оказывать сильную нагрузку на кэши
- Отдельно выделяют кэш инструкций (IC) и кэш данных (DC).
 - Нужно следить за своевременной инвалидацией IC

Кэши и multicore

Когда пишем?

- Каждый load загружает значение из кэша
- А что делать со store?
 - Память меняется!
 - Или нет?....
- Поведение определяется **политикой записи**



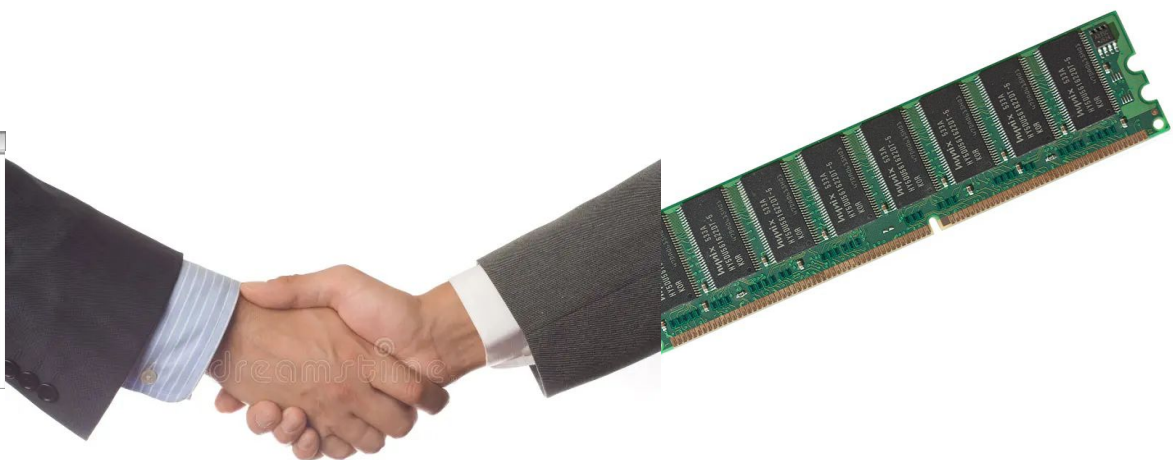
Политики записи

- Когда обновление придет в память?
- **Write Through** - одновременная запись в память и кэш
 - +: согласованность памяти и кэша
 - -: высокая нагрузка на шину памяти
- **Write Back** - обновление при вытеснении линии
 - +: низкая задержка store, низкая нагрузка на шину памяти
 - -: риск потери данных, требует проверки когерентности (snoops)
- **Write Combining** - накопление буфера для записи для сброса в память
 - +: повышение средней скорости записи
 - -: не гарантирует порядок доступов в память
- **Uncacheable** - запрет кэширования
 - Применяется для периферии



Согласны?

- Доступ к памяти имеет сразу несколько процессоров
- Требуется обеспечить согласованность данных, лежащих в кэшах разного уровня с реальным состоянием ОЗУ
- **Модели согласованности** - контракт между программами и памятью
- Набор правил, при соблюдении которых программой, работа памяти **предсказуема**

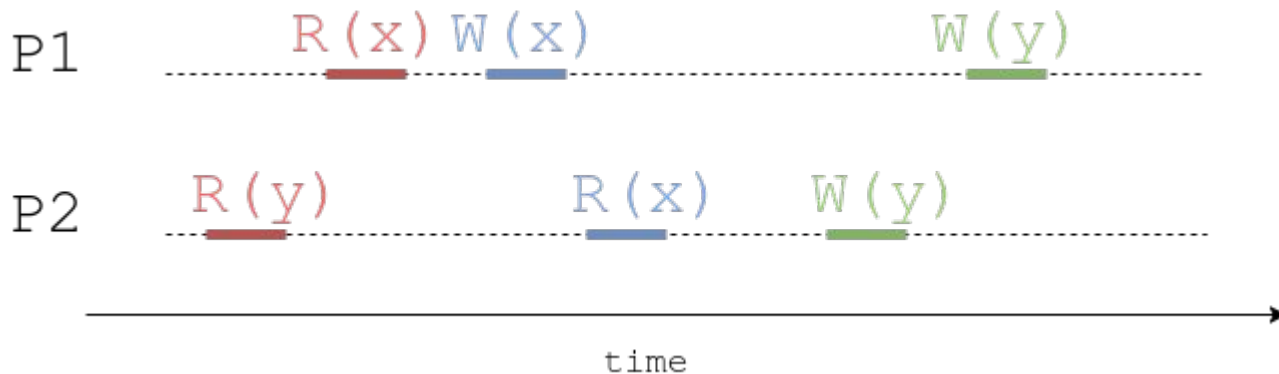


The image shows a hand in a dark suit sleeve holding a green RAM module, shaking hands with a computer code editor window. The code editor window is titled 'main.cpp' and contains the following C++ code:

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      cout << "Hello world!" << endl;
8      return 0;
9  }
10
```

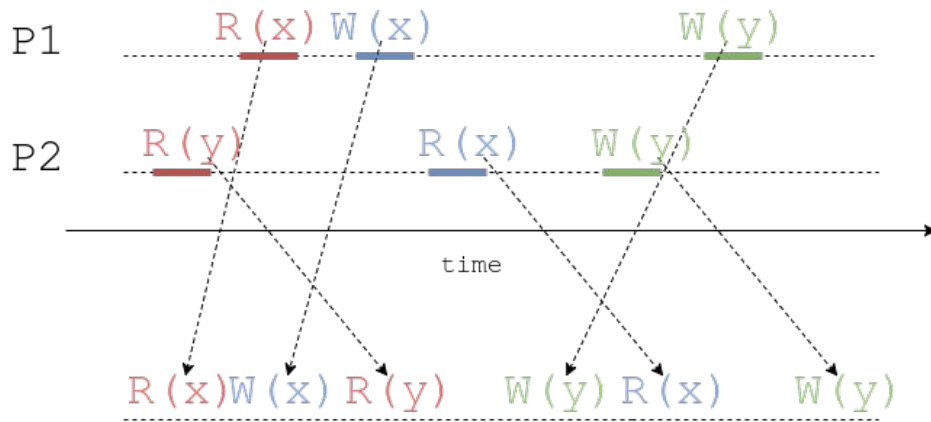
Строго

- Самая строгая модель - strict consistency
- Подразумевает наличие абсолютного времени
- Любая запись **моментально** обнаруживается всеми читателями
- Конкурентная запись **невозможна**



Последовательно

- Последовательная (sequential) согласованность
- Не требует глобального времени
- Порядок операций внутри процесса сохраняется
- Глобальный порядок не определен



Причинно

- Причинная (causal) согласованность
- Ослабляет последовательную согласованность
- Обеспечивает порядок только **зависимых** операций

P1 $W(x=1)$

P2 $R(x=1)$ $W(x=3)$

P3 $R(x=1)$ $R(x=3)$



P4 $R(x=3)$ $R(x=1)$



P1 $W(x=1)$

P2 $W(x=3)$

P3 $R(x=1)$ $R(x=3)$



P4 $R(x=3)$ $R(x=1)$

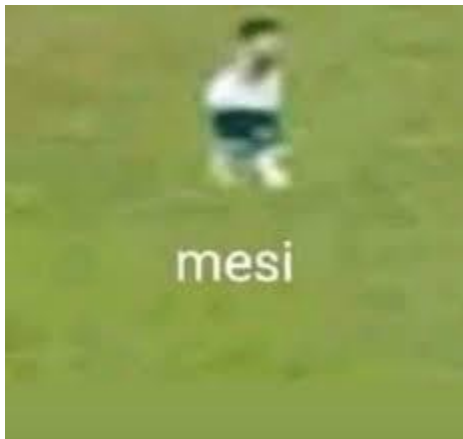


Что то еще?

- weak ordering
 - требуют явной синхронизации
- relaxed ordering
 - дальнейшее ослабление sequential consistency

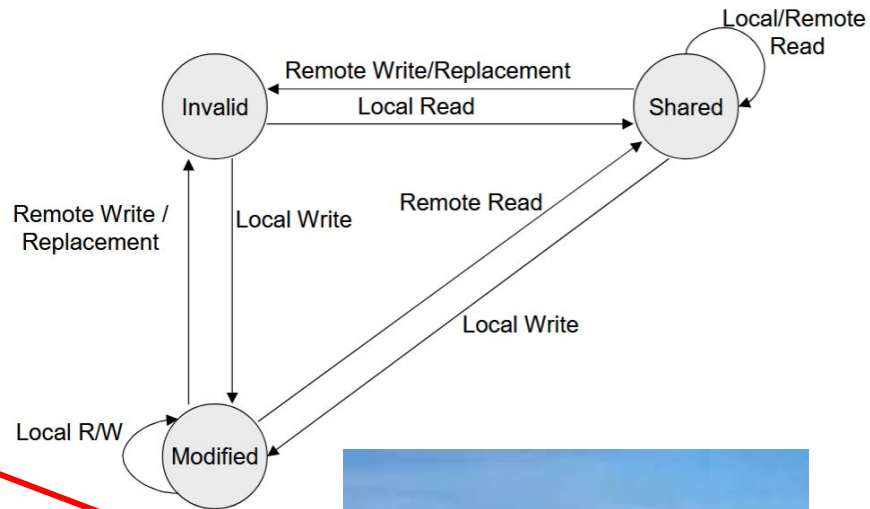
Протоколы когерентности

- Разобрались с моделями согласованности
- Снова вспоминим про кэши...
- Как поддержать модель согласованности?
- На помощь приходят **протоколы когерентности**
- Большинство современных протоколов есть вариации протокола MESI



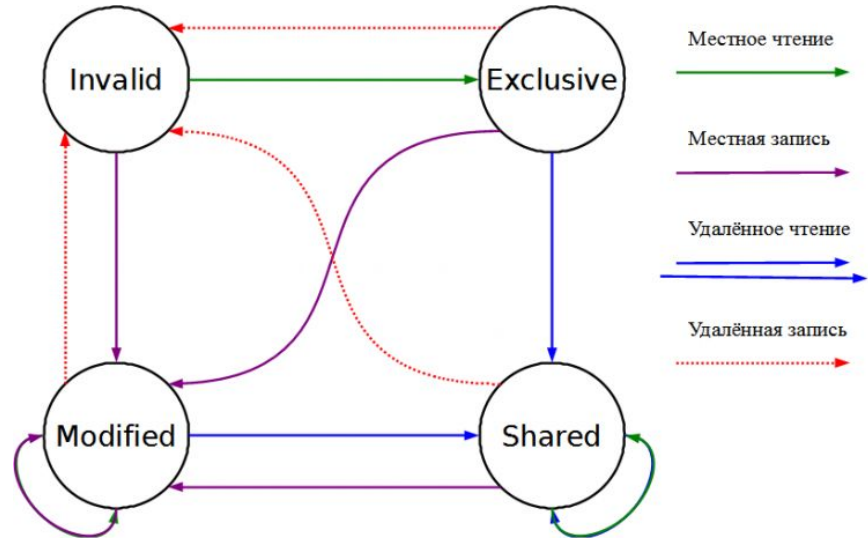
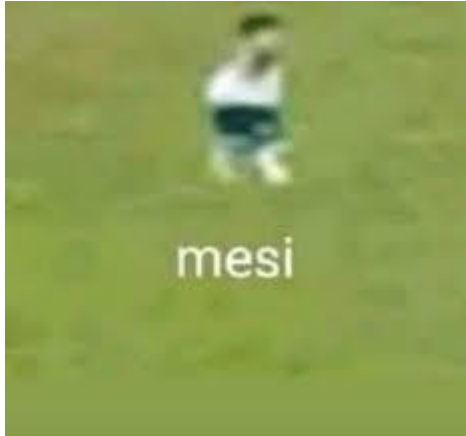
Протоколы когерентности. MSI

Time	P1	State	Bus Rq
0	-	I	-
1	Read X	S	read miss
2	Write X	M	invalidate



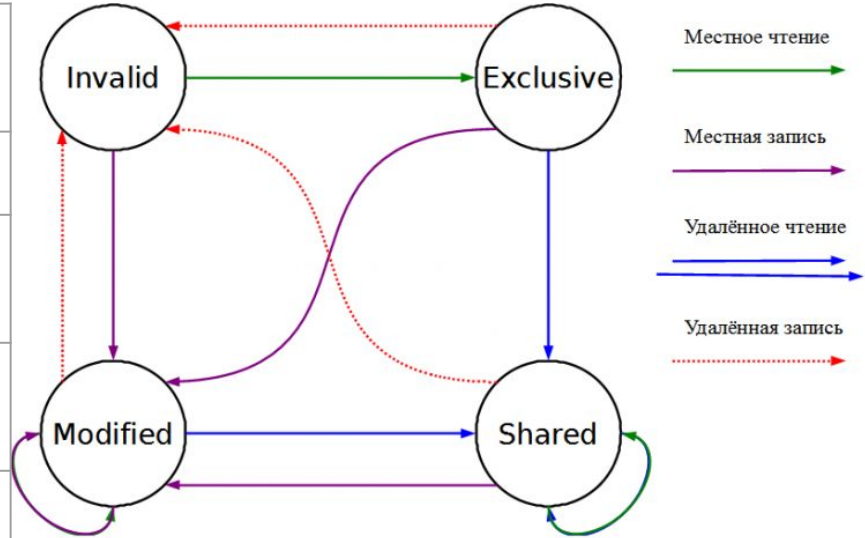
Протоколы когерентности. MESI

- Каждая линия имеет только одно из состояний
 - М - модифицированная, находится только в одном кэше. Может быть записана
 - Е - эксклюзивная, как М, только данные идентичны ОЗУ
 - S - разделяемая на несколько кэшей. При записи всегда идет запрос на шину
 - Все записи помечаются I
 - I - невалидная линия (либо пустая)



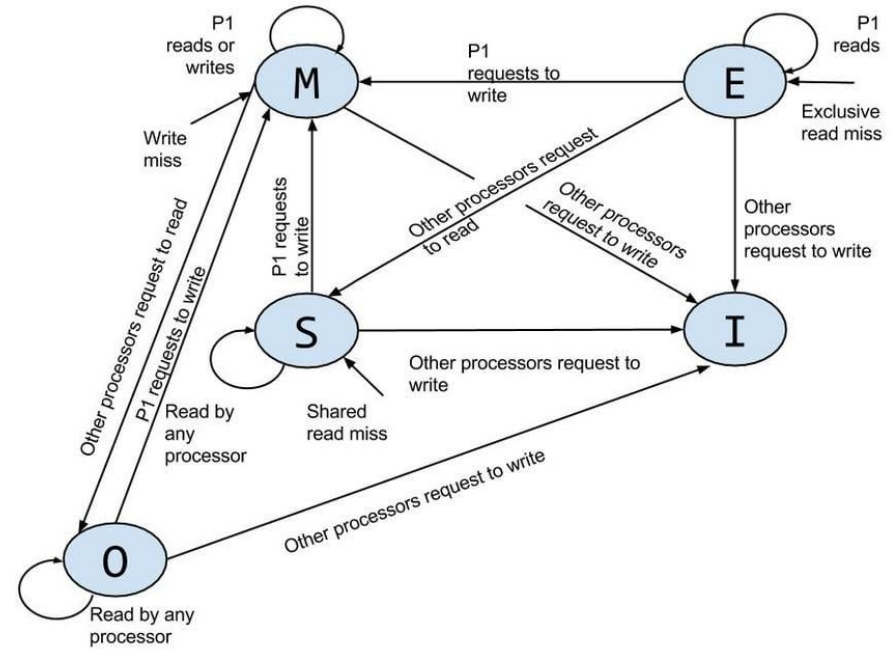
Протоколы когерентности. MESI

Time	P1	P1	State P1	State P2	Bus Rq
0	-	-	I	I	-
1	Read X	-	S	I	read miss
2	Write X	-	M	I	inv
3	-	Read X	S	S	read miss



Протоколы когерентности. MOESI

- Добавляет (еще) одно состояние
- О - линия содержит новые данные, которые могут отличаться от данных в памяти
- Позволяет читать из нее другим процессорам
 - Останавливает доступ к памяти



Можно еще?

- На самом деле да
- MESIF
 - + Forward
- MERSI
 - + Read-Only, or Recent
- Можно и проще, но не надо...
 - VI - Valid/Invalid

Влияние на симуляцию

- Подключим модель кэшей к функциональному симулятору
- Как изменится скорость?
 - Сильно упадёт - симулятор стал по сути потактовым
- Можно подключать симуляцию кэшей только на время изучения приложения
 - Кэши будут пустыми
- Требуется “разогревать” кэши (warmup)

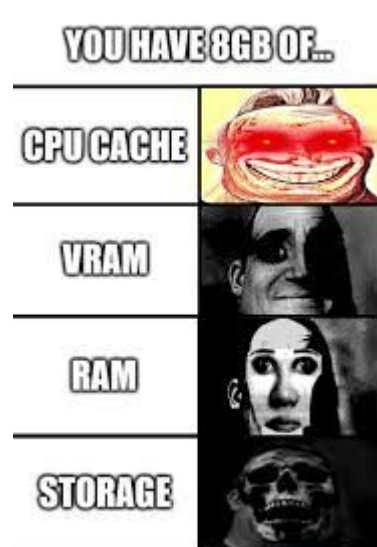
Загрузка ОС. Кэши
отключены

Прогрев кэшей. Кэши
включены. Измерений
нет

Изучение приложения.
Измерения включены

Кэшируем

- Важная часть CPU
- Позволяет ускорить работу с памятью
 - Которая сильно медленнее CPU
- Оптимальная политика замещения может зависеть от конкретного сценария
- В многопроцессорных системах нужно еще поддерживать когерентность
- При симуляции важно помнить про “прогрев” кэшей



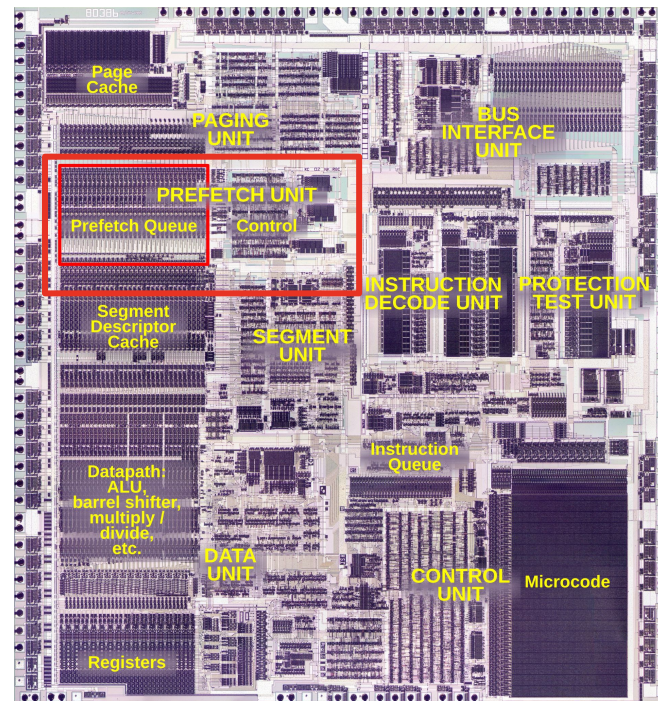
Предварительная выборка

Prefetching

- Данные могут попадать в кэш не только при первом обращении
 - Альтернатива – механизм предварительной выборки данных
 - Данные могут быть загружены в кэш до того, как процессор их запросит
- Предварительная выборка (англ. prefetching) – прогнозирование будущих обращений к памяти
 - Корректный prefetching исключает потенциальных промах в кэш память
- Кто и каким образом предварительно и любезно загружает данные в кэш еще до запроса?
- Давайте разбираться

Аппаратный prefetching

- Обычно встроен непосредственно в микроархитектуру процессора
 - Предварительная выборка кода происходит с помощью предсказателя переходов
- Существуют различные алгоритмы работы аппаратной предвыборки
- Чаще всего работает на уровне L2
 - Для L1 используются схемы с высокой точностью



i386 Die.

Prefetch Unit выделен красным прямоугольником

Программный prefetching

- Реализуется на уровне ISA с помощью специальных инструкций:
 - x86-64: PREFETCHH, где h – это хинты о локальности данных: T0, T1, T2, NTA.
 - ARM: PRFM
 - RISC-V: PREFETCH.##
- Оптимизация работает в связке с компилятором:
 - Компилятор анализирует циклы/паттерны доступа и вставляет prefetch-инструкции автоматически

```
for (int i = 0; i < N; ++i)
    res += a[i] * b[i];
```

Цикл без префетчинга

```
const size_t PRF_DIST = 16; // 64 bytes ~ cacheline
```

```
for (int i = 0; i < N; ++i) {
    if (i + PREFETCH_DIST < N) {
        // prefetch data after PRF_DIST ops num
        __builtin_prefetch(&a[i + PREFETCH_DIST], 0, 1);
        __builtin_prefetch(&b[i + PREFETCH_DIST], 0, 1);
    }
    res += a[i] * b[i];
}
```

Цикл с наивным префетчингом

Hardware Prefetching Algorithms

- Существует множество различных алгоритмов, позволяющих предсказать будущие обращение в память
- Принцип работы алгоритмов:
 - Выявление регулярных последовательностей (логично...)
 - “Угадывание” запросов в память через Bus Interface Unit (BIU)
- Алгоритмы:
 - Последовательная предвыборка (sequential)
 - Предвыборка с постоянным шагом (stride detection)
 - Обнаружение потоков (stream detection)
 - ... и многие другие

Предвыборка с постоянным шагом

- Простейший алгоритм предзагрузка следующих подряд линий
- Идея:
 - Обнаруживает обращения к последовательно растущим (или убывающим) адресам
 - При обращении к адресу, расположенному в `t1`:

```
lw t0, 0x0(t1)
```

- Префетчер загружает одну или несколько следующих кэш-линий (+64 байта)

```
prefetch 0x40(t1)
```

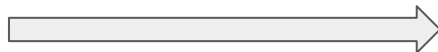
- Глубина префетч механизма может быть фиксированной или адаптивной
- +** Отлично подходит для предзагрузки инструкций, линейный массивов
- Не работает при случайном доступе или нерегулярных паттернах

Последовательная предвыборка

- Алгоритм предсказания доступа с постоянных шагом
- Идея:
 - Железка анализирует последовательность адресов и определяет постоянное смещение (шаг, он же stride) между обращениями:

```
0x00402f0: lw t0, 0x20(t1)
0x00402f4: add t2, t2, t0
0x00402f8: addi t1, t1, 0x100
0x00402fc: beq t1, t3, -4
```

stride = 0x100



```
prefetch 0x00f4280
prefetch 0x00f4380
...
```

- Необходимо хранить истории адресов для определенного контекста
- + Эффективен для структур с фиксированным шагом
- Аппаратные затраты для хранения памяти истории

Выборка путем обнаружения потоков

- Алгоритм выделения независимых потоков обращений
 - Поток – последовательность обращений к памяти, демонстрирующая регулярность
 - Идея:
 - Процессор выделяет несколько активных потоков и ведёт для каждого отдельный prefetch
 - Схоже на stride detection, однако не обращается к РС
 - Используются stream buffers – небольшие аппаратные очереди, хранящие предвыбранные линии для каждого потока.
- +** Эффективен при одновременной работе с несколькими структурами данных
- Требуется значительных аппаратных ресурсов (буферы, логика сравнения)

Типы предварительной выборки

Программная

Реализуется в ISA

Требует явных
инструкций

Знания разработчика о
паттернах доступа

Увеличение размера
кода

Аппаратная

Реализуется в uArch и
железе

Работает
автоматически

Статистика обращений
в память

Замусоривание кэша